The type parameters are the same generics that modules and classes have, and the type at the end is any RBS type as defined in RBS Types, on page 489.

In the RBS documentation, constants and globals are considered declarations, and they both work the same way with the name followed by a colon followed by the type:

constant: *type*
$*global*: *type*

As in:

```
class Post
  DEFAULT_TITLE: String
end

$ADMIN_EMAIL: String
```

RBS Members

An RBS member is most of what you would think of as the contents of a class. Unlike regular Ruby, RBS expects you to declare the type of instance variables. Instance variables that are not declared will be type errors as far as RBS tools are concerned:

@instance_variable: *type*

If you are using attr_accessor and its sibling methods, you can declare the type of the internal variable. By default, this also declares an instance variable of the same name, but you can override that by putting a different variable name in parentheses, or you can omit declaring a variable name by including empty parenthesis (presumably because the variable name has already been declared).

attr_*attribute_type name* : *type*
attr_*attribute_type name* (*ivar*) : *type*
attr_*attribute_type name* () : *type*

Any of the attr methods can be used here:

```
class Person
  @id_number: String
  attr_accessor name: String
  attr_reader height (@height_in_inches): Integer
  attr_writer id_number (): String
end
```

Most of the body of RBS classes are going to be method names where you are specifying the types of the arguments and the type of the return value. This gets a little tangled because Ruby has a lot of different ways to define method parameters, but the basic idea is that every element of the method definition can have a type decoration.

The simplest version of the syntax is the method name, followed by a colon, followed by all the parameters, followed by a skinny arrow, followed by the return type:

def *method_name*: (*params*) -> *type*
def *method_name*: (*params*) -> *type*‹ | (*params*) -> *type*›